

CPU nur wenige Register besitzt und diese Register wiederverwendet werden müssen. Um die Register wiederzuverwenden, muss der enthaltene Wert zurück in den Speicher geschrieben werden. Dies ist in der höheren Programmiersprache nicht ersichtlich und wird erst sichtbar, wenn man sich den Assembler-Quellcode ansieht, den der Compiler erstellt.

2.3 Matrixmultiplikation mit Strip-Mining und SIMD

Ein Ansatz diesen Verlust, der durch Load-Store entsteht zu minimieren, ist das Strip-Mining in Kombination mit der Verwendung von SIMD.

SIMD (Single Instruction Multiple Data) erlaubt es, mehr als ein Datum gleichzeitig durch eine einzige Instruktion verarbeiten zu können. Die Addition zweier Vektoren geschieht aus mathematischer Sicht elementweise und jedes Elementpaar kann unabhängig von allen anderen Paaren in jeglicher Reihenfolge addiert werden. Damit eignet sich die Vektoraddition für SIMD Anweisungen. Dazu muss die CPU die passende Hardware bereitstellen. Dann ist es möglich, dass die CPU, wenn eine Vektoradditionsanweisung aufgeführt wird, gleich mehrere Elemente auf einmal miteinander addiert.

Abbildung 2.1 stellt SISD auf der linken und SIMD auf der rechten Seite schematisch dar. SISD addiert pro Aufruf ein Elementpaar, wobei SIMD mehrere Paare pro Aufruf addiert.

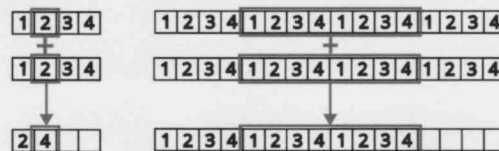


Abbildung 2.1: SISD (links) und SIMD (rechts)

Des Weiteren erlaubt das Strip-Mining auch Datenobjekte zu verarbeiten, die so groß sind, dass sie nicht als Ganzes in die Register der CPU passen.

Beim Strip-Mining wird von einer Ressourcen-Hierarchie ausgegangen. Die Ressourcen sind dabei die Register und der Speicher einer CPU, wobei Register letztendlich auch nur sehr kleine Speicherzellen sind. Der Grundgedanke ist, dass je näher Speicher am CPU liegt, desto dessen Preis steigt und daher kleiner in der verfügbaren Größe ausfallen muss, aber gleichzeitig auch um einiges schneller als ein weiter entfernt liegender Speicher ist. Die Hierarchie besteht aus den Registern, gefolgt von einem oder mehreren Cache-Leveln und schließlich den RAM-Bausteinen.

Die Vektor- oder Matrix-Daten kommen aus dem Speicher und müssen dem CPU in Form seiner Register zugeführt werden. Die Addition zweier Vektoren beispielsweise kann nur durchgeführt werden, wenn Vektordaten in den Registern der CPU stehen. Die Kunst besteht also darin, die Daten in effizienter Form an den CPU heranzutragen. Beispielsweise ist es ineffizient ständig im

Speicher hin- und her zu springen. Ein lineares Laden ist effizienter um die Lokalität der Daten innerhalb des Cache auszunutzen.

Eine Cache-Hierarchy legt nahe, Vektoren blockweise zu laden, so das ein Block jeweils genau in den verfügbaren Cache passt. Einmal geladen, dient der Cache dem CPU als schnellen Speicher und ein Durchgriff auf den langsameren RAM-Speicher erfolgt nur, wenn die Daten im Cache verarbeitet worden sind.

Die Register innerhalb der CPU sind von begrenzter Größe. Eine sinnvolle Annahme ist etwa 1024 bit Registerbreite. Damit passen dort dann 32 Elemente eines Vektors hinein, wenn jedes Element eine 32-Bit Zahl darstellt.

Wenn nun sehr große Vektoren addiert werden sollen, dann kommt Strip-Mining zum Einsatz. Mit groß ist hier gemeint, dass die Vektoren weder im Ganzen in die CPU-Register passen noch passen die Vektoren im Ganzen in einen Cache-Frame. Die Software verwendet eine Instruktion um zu ermitteln, wie viel Platz innerhalb der CPU-internen Register verfügbar ist. Die CPU antwortet mit der Registerbreite.

Für die Berechnung von Vektoren wurde die RISC-V Vector Extension (RVV) [2] erstellt und ratifiziert. Die Vektor Extension ist für diese Arbeit relevant, obwohl sie zwar Vektoren verarbeitet, aber damit die Grundlage für die Verarbeitung von Matrizen legt.

2.4 Die RISC-V Vector Extension

Eine RISC-V CPU besitzt die sogenannte X-Register-File. Es handelt sich damit um alle Register, die eine RISC-V CPU standardmäßig, ohne Extensions hat. Dies könnten beispielsweise die Register x0 bis x31 sein. Die Länge der Register in bits wird durch die Konstante XLEN festgelegt.

Die RVV erweitert eine RISC-V CPU um zusätzliche Vektor Register v0 bis v31. Die Länge der Vektor Register wird durch die Konstante VLEN (in Anlehnung an die Konstante XLEN) vorgegeben. VLEN wird bei der Synthese des Designs auf einen Wert festgelegt und ist nicht zur Laufzeit änderbar. VLEN stellt somit eine Eigenschaft der CPU dar. Die RVV Spezifikation [2] macht die Vorgabe:

"The number of bits in a single vector register, $VLEN \geq ELEN$, which must be a power of 2, and must be no greater than 2^{16} ".

Das besondere an den Vektor Registern ist, dass sie konfiguriert werden können. Es lässt sich festlegen, welcher Datentyp in den Registern erwartet wird, also 8-bit Bytes, 16-bit Halbwörter oder 32- oder 64-bit Wörter. Zusätzlich lassen sich die Vektorregister Gruppieren, also aneinander hängen um noch größere Register zu bilden.

Der Vorteil ist, dass man beispielsweise 8 Bit Bytes als Datentyp festlegen

kann und dann vier Vektorregister gruppieren kann. Unter der Annahme dass ein RISC-V Chip mit einer VLEN von 2^{16} synthetisiert worden ist, ergibt sich damit Speicher für beispielsweise 262144 bytes. Werden zwei Vektoren dieser Länge in den CPU geladen, kann eine Vektoraddition all dieser Elemente mit nur einer einzigen Instruktion erfolgen! Dies ergibt einen Speedup um den Faktor 262144 gegenüber einem naiven Load-Store Ansatz, bei dem jedes 8-Bit Elementpaar einzeln geladen und verrechnet wird.

LR, LS
LV-
-
LR-

Die Abbildung 2.2 aus [9] zeigt zwei mögliche Konfigurationen. Beide Konfigurationen gruppieren die vier Vektorregister V4 bis V7. Die obere Konfiguration addiert 64 Bit Datentypen, während die untere Konfiguration 32 Bit Datentypen addiert. Die Vektor-Engine wird basierend auf der aktuellen Konfiguration automatisch die korrekten Stellen der Register auslesen und miteinander addieren.

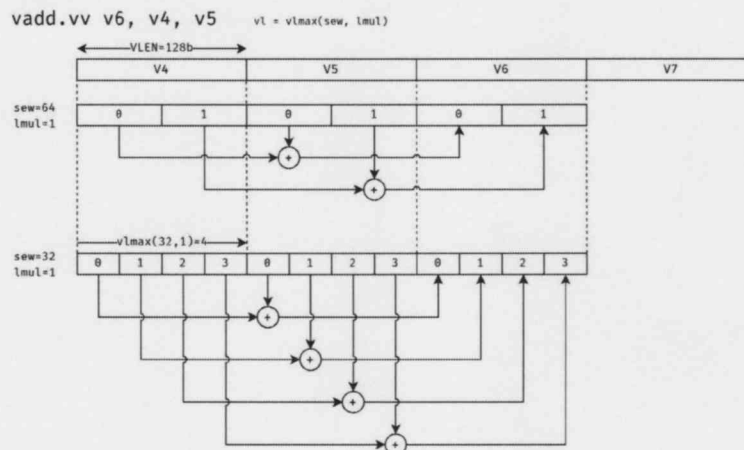


Abbildung 2.2: RVV Register

LV-

2.5 Das Hardware-Software-Co-Design

Die Verwendung der Vector-Extension erfolgt durch ein Hardware-Software-Co-Design. Damit ist gemeint, dass eine Anwendung mit der Hardware zusammenarbeiten muss um von der Hardware-Beschleunigung zu profitieren. Es ist weder möglich durch Software oder Hardware alleine eine Beschleunigung zu erzielen.

lk
- noch durch

Moderne, optimierende Compiler wie der GCC oder die LLVM Infrastruktur wurden um das Wissen um die RISC-V Vektor-Extension erweitert und können den Anwendungsfall automatisch erkennen und das gewünschte Hardware-Software Co-Design automatisch generieren!

M-
LV-
-
Ln, Lx

Dies lässt sich testen indem man den naiven Matrix Multiplikationsalgorithmus mit der neuesten Version des GCC für RISC-V übersetzt. Im generierten

Assembler-Code findet man die RVV-Instruktionen wieder! Der Compiler hat den Anwendungsfall erkannt und das Co-Design erstellt.

Das Hardware-Software Co-Design sieht das iterative Vorgehen Strip-Mining vor, bei dem die Software die Hardware in jeder Iteration aufs neue konfiguriert und dann Daten zur parallelen Verarbeitung durchführt.

Ein Ablaufdiagramm ist in 2.3 dargestellt:

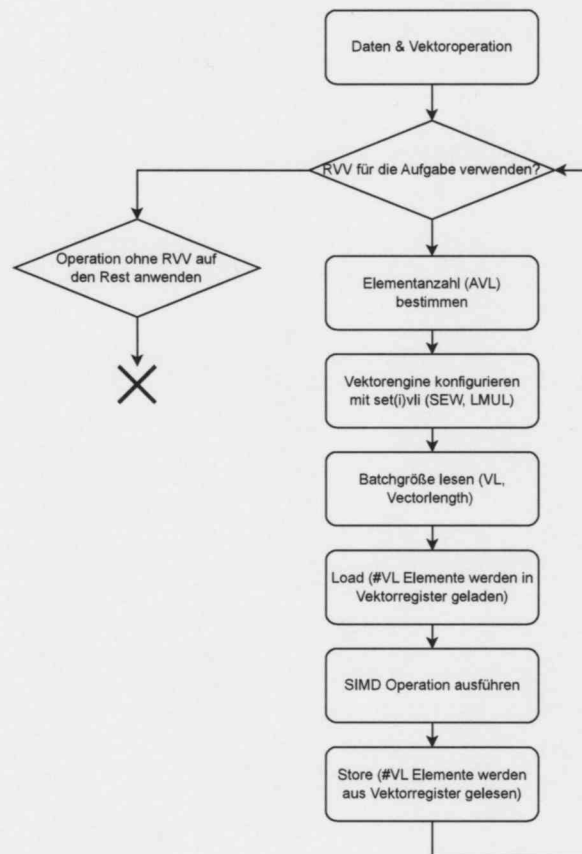


Abbildung 2.3: Strip-Mining Ablauf

Die Schritte sind:

1. Die erste bzw. nächste Iteration beginnt.
2. Die Applikation entscheidet, ob es effizienter ist die Vektoroperation mit oder ohne Vektor-Extension auszuführen. Wenn die Vektorlänge

beispielsweise nur aus zwei Elementen besteht ist es schneller diese beiden Element einfach so zu verrechnen.

3. Wenn sich die Anwendung dazu entscheidet die Vektor-Extension zu verwenden, dann beginnt die Strip-Mining-Schleife. Wenn es zu wenige Elemente gibt, dann springt die Anwendung zu einem Label, dass die Verarbeitung terminiert und berechnet den Rest der Element dort manuell ohne Hardwarebeschleunigungen.
4. Die Anwendung bestimmt den Anteil der Vektorelemente, die noch verarbeitet werden müssen. Dieser Wert wird Application Vector Length (AVL) genannt. Beispielsweise entspricht die AVL in der ersten Iteration der gesamten Länge des Vektors. Mit jedem Schleifendurchlauf werden Teile des Vektors verarbeitet und die AVL nimmt immer mehr ab bis schließlich keine Elemente zur Verarbeitung mehr übrig sind und das Verfahren terminiert.
5. Neben der AVL muss die Anwendung auch in jeder Iteration den Datentyp bzw. die Größe der einzelnen Vektorelemente konfigurieren. Die Größe wird (= Selected Element Width, SEW) genannt. Es wird auch bestimmt, wie die Vektorelement in den Registern angeordnet werden und wie die Vektorregister gruppiert werden. Diese Konfiguration wird Group Modifier oder LMUL genannt. Um die Konfiguration vorzunehmen ruft die Anwendung die Instruktion set(i)vli auf.
6. Die Vector Engine verwendet die konfigurierten Werte AVL, SEW und LMUL und berechnet die Batch Größe. Ein Batch ist die Menge an Vektorelementen die in einer einzigen SIMD-Operation verrechnet werden. Hier kommt der Hardware-Teil des hardware-software co-designs zum tragen, denn der RISC-V CPU und die enthaltene Vector-Engine müssen entscheiden, wie viele Elemente jetzt tatsächlich parallel verarbeitet werden können. Dabei berücksichtigt die Vector-Engine wie viele Rechenknoten sie zur Verfügung hat um parallel zu rechnen. Die ermittelte Anzahl an Elementen wird vector length (VL) genannt und von der Hardware an die Anwendung zurückgegeben. Dazu gibt die Instruktion set(i)vli diesen Wert in ein Rückgaberegister aus, aus dem sich die Anwendung den Wert holen kann.
7. Die Anwendung kennt jetzt die VL, die für diese Iteration gilt. Damit kennt die Software die Batch-Size. Die Software wird nun die Vector-Load Instruktionen (vle8, vle16, ...) verwenden um einen Anteil des Vektors oder der Vektoren, der genau der VL entspricht in die Vektorregister zu laden. Die Anwendung wird auch einen Zeiger nach vorne verschieben, um sich zu merken von welcher Stelle in der nächsten Iteration geladen werden muss.
8. Die Vector-Engine verarbeitet die Vector-Load Anweisungen und schreibt die geladenen Daten in die Vektorregister.
9. Die Anwendung führt nun eine Instruktion aus um die Vektoren zu verrechnen. Es können beispielsweise eine elementweise Addition (vadd),

Lt, Lr,
Lte

1st Lte
Satzbau Verb
fehler

Lb,

Lte

Lr-
Lh-
Ln, 1H1C
IT
Le,

Ik Lt,
Ln,

Ld-

Subtraktion (vsub), Multiplikation oder sonstige Operationen ausgeführt werden.

10. Die Vektor-Engine führt nun die Instruktion auf allen Elementen parallel aus. Es handelt sich also um eine SIMD-Operation, bei der mehrere Daten aufgrund einer einzigen Anweisung verarbeitet werden. Damit ist der aktuelle Batch der Iteration abgearbeitet.
11. Die Anwendung führt eine Store-Anweisung aus um die Ergebnisdaten aus dem Ergebnisregister in den Speicher zurückzuschreiben und Platz für neue Ergebnisdaten zu schaffen.
12. Die Vektor-Engine verarbeitet die Store-Anweisung und die Daten werden in den Cache bzw. den RAM übertragen.
13. Die nächste Iteration beginnt. Im Grunde springt die Anwendung von hier zurück zum ersten Punkt dieser Auflistung.

Es folgt ein Beispiel in RISC-V Assembly, das das obige Vorgehen umsetzt. Die Anweisungen aus der Vektor-Extension sind weniger eingerückt als die normalen RISC-V Anweisungen.

```

1 // vector-vector add routine of 32-bit integers
2 //
3 // void vvaddint32(size_t n,
4 //     const int*x, const int*y, int*z)
5 // {
6 //     for (size_t i=0; i<n; i++)
7 //     {
8 //         z[i] = x[i] + y[i];
9 //     }
10 // }
11 //
12 // Mapping parameters to argument register (a0 .. a7)
13 // a0 = n, a1 = x, a2 = y, a3 = z
14 //
15 // Non-vector instructions are indented
16 vvaddint32:
17
18     // Set vector length based on 32-bit vectors.
19     // t0 contains the amount of elements that will be
20     // processed
21     vsetvli t0, a0, e32, ta, ma
22
23     // Load for first vector
24     vle32.v v0, (a1)
25
26     // Decrement number of elements that
27     // still need processing.
28     // t0 is the result of vsetvli. It
29     // is the amount of elements processed
30     // this iteration
31     sub a0, a0, t0
32
33     // Multiply number done by 4 bytes (= 32 bit
34     // integer) because of SEW=e32
35     slli t0, t0, 2
36
37     // Increment/Advance pointer for first vector

```

lc
Bitte entscheiden
Vector-
Engine
oder:
Vektor-
Engine

```

38    add a1, a1, t0
39
40    // Load for second vector
41    vle32.v v1, (a2)
42    // Increment/Advance pointer for second vector
43    add a2, a2, t0
44
45    // Sum vectors (with hardware acceleration)
46    vadd.vv v2, v0, v1
47
48    // Store result back to memory
49    vse32.v v2, (a3)
50
51    // Increment/Advance pointer for destination
52    add a3, a3, t0
53
54    // Loop back to the beginning of the for loop
55    // if the element count is not zero. Otherwise
56    // continue with the ret instruction
57    bnez a0, vvaddint32
58
59    // Finished with the function
60    ret

```

2.6 Übergang zur einer Matrix Extension

Jetzt, da die Wirkungsweise der RVV Extension für Vektoren beschrieben worden ist, kann nachvollzogen werden, in welchem Spannungsfeld die RISC-V Gremien Entscheidungen treffen müssen, wenn es um den nächsten Schritt hin zu einer Matrix-Extension geht.

Da die RVV Extension ratifiziert worden ist, gibt es bereits Chips, die auf dem Markt verfügbar sind und die RVV Extension implementieren. Ein Beispiel ist der OrangePi RV2 Single Board Computer, der einen Spacemit-K1 RISC-V Chip verwendet.

Das bedeutet, Firmen haben bereits Designs erstellt und Investitionen in die RISC-V RVV Strategie investiert. Wenn eine Matrix-Extension Vektorregister wiederverwenden würde, hätten solche Firmen einen Vorteil. Auf der anderen Seite möchte man das bestmögliche Verfahren ermöglichen um effizient mit Matrizen rechnen zu können und sich damit von der Konkurrenz abzuheben. Eventuell ist der Ansatz, der für Vektoren gewählt wurde, für Matrizen nicht optimal und es ist lohnenswerter komplett neue Wege zu gehen und auf die Wiederverwendung von Hardware zu verzichten.

Es gibt laut Krste Asanovic [4] bei der Arbeit an einer Extension für Matrizen drei große Strömungen innerhalb der RISC-V Organisation. Für jeden der Ansätze wurde ein Charter gebildet und alle Charter treiben ihre Arbeit eigenständig voran.

Zunächst wird in der linken Spalte der Abbildung 2.6 gezeigt, dass sich eine Matrix theoretisch elementweise berechnen lässt, wenn für jedes Element der Matrix individuell ein Skalarprodukt mit Hilfe der RVV-Extension berech-

LV-

LV-

LV-

LV-

LV-

LV-

LI

LV-

LI

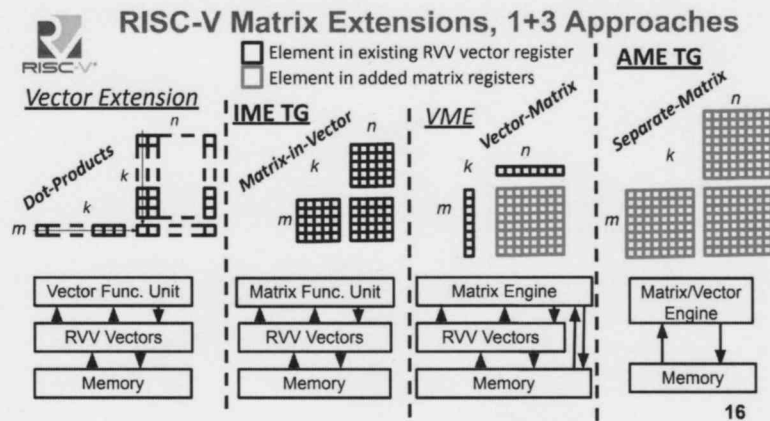


Abbildung 2.4: Ansätze zur Matrix-Extension

net wird. Dies ist nur möglich, wenn die CPU die RVV besitzt und gleichzeitig, wenn die Matrix spalten- oder zeilenweise in ein Vektorregister passt.

Die Integrated Matrix Extension IME [10] verwendet die RVV-Register wieder um darin Matrix-Berechnungen auszuführen. Es wird keine spezielle Hardware für die Matrix-Berechnung hinzugefügt. Die erklärten Ziele des IME-Charter sind es, einen bedeutenden Geschwindigkeitsvorteil zu erzielen, bei vergleichbaren Kosten, wie sie bereits für die RVV anfallen.

Die Vector-Matrix Extension VME baut auf die RVV-Extension auf und führt zusätzlich einen sogenannten Matrix-State bzw. eine Matrix-Engine hinzu. Mit dem Matrix-State ist gemeint, dass die CPU um Komponenten erweitert wird, die in der Lage sind Zwischenergebnisse bei einer Matrix-Multiplikation zu speichern. Es kommt bei der VME also weitere Hardware für die Matrix-Berechnung hinzu. Diese zusätzliche Hardware ist in der Abbildung in roter Farbe markiert.

Die Attached Matrix Extension AME geht noch einen Schritt weiter und verwendet dabei keine Vektorregister aus der RVV-Extension mehr, sondern fügt Architecture State sogar für komplette Matrix Register hinzu. Sowohl die Eingabedaten als auch die Zwischen- und Endergebnisse werden in speziellen Matrix-Registern gespeichert. Damit ist die AME als einziger Ansatz unabhängig von der RVV-Extension und könnte auch komplett ohne RVV-Extension angeboten werden. Auch hier ist neue Hardware in roter Farbe dargestellt.

Für die offizielle Ratifizierung einer RISC-V Matrix-Extension ist zum jetzigen Zeitpunkt nicht klar, welcher Ansatz verwendet wird oder ob eventuel sogar mehr als eine Spezifikation ratifiziert werden wird.

In dieser Bachelor-Arbeit wird ein Ansatz verfolgt, der sich vom Architecture State her am ehesten an der Attached Matrix Extension AME orientiert. Die Entscheidung fiel zugunsten der AME orientierten Lösung, da die NEORV32 CPU im jetzigen Zustand keine RVV-Extension implementiert und

daher zunächst RVV Register erstellt werden müssten um IME oder VME zu implementieren. Ein weiterer Grund ist, dass sich die zusätzliche Matrix-Hardware der AME sehr logisch aus der Berechnung des Outer-Produkts ergibt und daher ein einfaches Verständnis der zu erledigenden Aufgaben möglich ist. Im begrenzten Zeitrahmen dieser Bachelorarbeit wurde daher zugunsten einer Lösung entschieden, die intellektuell auch unter Kontrolle gehalten werden kann und für die ein Abschluss der Arbeit absehbar ist.

L11

Es werden Instruktionen für die Matrixberechnung vorgesehen, wie sie auch bei der Vektor-Extension zu finden sind. Damit ist gemeint, dass die hinzugefügten Instruktionen den Strip-Mining Ansatz ermöglichen. Daher werden Load- und Store-Anweisungen und eine Anweisung für die Operation der Matrixmultiplikation vorgesehen. Damit kann eine Matrixmultiplikation auch für sehr große Matrizen, die nicht als Ganzes in die CPU-Register passen, erfolgen. Außerdem können so auch Kernel in der blis Bibliothek implementiert werden.

L5-

2.7 Blockweise Matrixmultiplikation

Im Folgenden wird das blockweise Vorgehen erläutert, mit dem die Matrixmultiplikation ausgeführt werden wird.

Bei der herkömmlichen Matrixmultiplikation über das Skalarprodukt von Zeilen und Spalten werden die beiden Matrizen A und B in ihrer kompletten Dimension betrachtet, da jeweils komplette Zeilen und Spalten verrechnet werden müssen um ein Matrix-Element zu bestimmen.

Lk

Bei sehr großen Matrizen ist dies nicht zielführend, da die CPU die Matrix nur schrittweise erfassen und damit nur sehr ineffizient berechnen kann.

Die angestrebte Verbesserung besteht darin, eine Matrix blockweise zu berechnen. Die Abbildung 2.5 visualisiert das blockweise Vorgehen.

L5

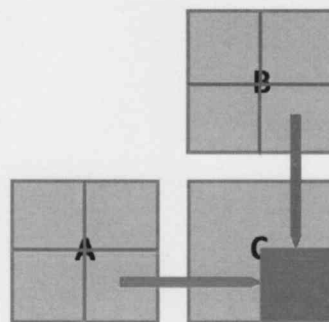


Abbildung 2.5: Matrix in Blöcken

Die Matrizen müssen nicht mehr im Ganzen in den CPU oder Cache passen sondern es kann nur ein Teil der Matrizen betrachtet werden.

L11

L11

oder: Die Matrizen müssen

Zusätzlich eröffnet sich ebenfalls das Potential der parallelen Berechnung. Wie bei Vektoren ist es durch das blockweise Vorgehen auch für die Berechnung von Matrizen egal in welcher Reihenfolge die einzelnen Ergebnisse berechnet werden. Damit ist auch eine vollkommene Parallelität denkbar.

Als Ausblick würde das für ein RISC-V Design bedeuten, dass ein RISC-V CPU aus mehreren Harts synthetisiert wird, wobei jeder Hart eine Matrix-Engine erhält, also in der Lage ist, die Matrixmultiplikation hardwarebeschleunigt zu berechnen. Dann wird eine große Matrix auf alle Cores verteilt um das Ergebnis in einzelnen Blöcken zu berechnen und zurück in den RAM-Speicher zu schreiben.

Zunächst wird anhand eines Beispiels verdeutlicht, was es bedeutet, wenn von blockweiser Berechnung gesprochen wird.

A C	B D	E G	F H
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16
17	18	19	20
21	22	23	24
25	26	27	28
29	30	31	32

Abbildung 2.6: Matrixmultiplikation blockweise

In Abbildung 2.6 wird die grüne Matrix mit der blauen Matrix multipliziert. Es werden einzelne Blöcke A bis H innerhalb der beiden Matrizen definiert.

Zur Gegenprüfung ist das Ergebnis in Abbildung 2.7 dargestellt.

250	260	270	280
618	644	670	696
986	1028	1070	1112
1354	1412	1470	1528

Abbildung 2.7: Matrixmultiplikation Ergebnis

Dieses Ergebnis wird den Berechnungsvorschriften aus 2.8 zufolge berechnet. Beispielsweise gilt für den grauen Block die Vorschrift $A * E + B * G$

A	B	E	F
C	D	G	H
nach links		nach unten	

$A * E + B * G$
 $C * E + D * G$

$A * F + B * H$
 $C * F + D * H$

Abbildung 2.8: Matrixmultiplikation Berechnungsvorschrift

Der grau markierte Block ist das Ergebnis der Multiplikation der Blöcke A, E, B und G. Es ist zu prüfen, dass die Multiplikation der Blöcke tatsächlich den grauen Bereich der Ergebnismatrix ergibt.

In Abbildung 2.9 wird der grau hervorgehobene Bereich der Berechnungsvorschrift zufolge berechnet. Zunächst werden die Blöcke A mit E und B mit G zusammen sortiert. Das Ergebnis der beiden Multiplikationen ist in der folgende Zeile zu finden. Das Ergebnis der Addition der Zwischenergebnisse in der untersten Zeile. Die unterste Zeile enthält den grauen Block.

Mit dem Wissen über die blockweise Aufteilung ist es nun möglich per SIMD zu parallelisieren oder Strip-Mining auszuführen, wenn keine parallelen Rechenknoten verfügbar sind. In dieser Arbeit werden Instruktionen analog zur

2 Theoretische Grundlagen

A	E	B	G
1	2	3	4
5	6	7	8
17	21	25	29
18	22	26	30

A*E	B*G
59	191
211	407
62	198
222	422

A*E + B*G
250
618
260
644

Abbildung 2.9: Matrix Multiplikation Blöcke A, E, B und G

RVV bereitgestellt um das Strip-Mining für Matrizen zu ermöglichen und dabei blockweise vorzugehen.

Es ist noch zu klären, wie die einzelnen, inneren Blöcke miteinander multipliziert und addiert werden. Die Instruktionen, die im Rahmen dieser Arbeit in die NEORV32 CPU eingebracht werden, ermöglichen genau die Berechnung dieser inneren Blöcke. Der Rest muss durch ein Hardware-Software Co-Design organisiert werden.

2.8 Outer-Produkt

Das Outer-Produkt wird verwendet um die Matrixmultiplikation von Blöcken auszuführen. Die Eigenschaft, die das Outer-Produkt nützlich macht ist, dass es die Matrixmultiplikation auf die Berechnung von Additionen und Multiplikationen in einer gleichförmigen Form reduziert. Damit lässt sich das Verfahren einfach in einer Hardwarebeschreibungssprache formulieren. Die Synthese-Werkzeuge können dann parallelisierte Hardware zur Berechnung generieren.

Das Outer-Produkt Verfahren erhöht auch die Parallelität im Vergleich zur Berechnung einzelner Zellen der Ergebnismatrix über einzelne Skalarprodukte. Beim Outerprodukt wird ein gesamtes Feld pro Durchgang berechnet. Nach allen Durchgängen enthält das Feld dann die Ergebnismatrix der Matrixmultiplikation.

In der Übersicht [11] von Earl A. Killian wird das Outer-Produkt auf Seite 53 beschrieben.

"The outer product is a fully parallel computation of mn multiplications."

Das Outer-Produkt wird aus zwei Vektoren u und v berechnet.

$$u = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix}, v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

Das Symbol ist $\otimes$ und das Ergebnis ist eine $m \times n$ Matrix C . Die Rechenvorschrift ist:

$$C = u \otimes v = \begin{pmatrix} u_1 v_1 & u_1 v_2 & \cdots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \cdots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1 & u_m v_2 & \cdots & u_m v_n \end{pmatrix} \quad (2.2)$$

Nachdem das Outer-Produkt definiert ist, lässt sich das Outer-Produkt aufaddieren, um die Matrixmultiplikation zu beschreiben.

"Using this formulation, the matrix product can be expressed as the sum of K outer products of the columns of A with the rows of B "

$$C = AB = A_1^* \otimes B_1^1 + A_2^* \otimes B_2^2 + \dots + A_M^* \otimes B_N^N = \sum_{p=1}^k A_p^* \otimes B_p^p \quad (2.3)$$

In der Formel (2.3) sind A und B zwei Matrizen, die miteinander multipliziert werden. Man kann erkennen, dass es nicht ausreicht lediglich die Multiplikationen aus Formel (2.2) auszuführen, vielmehr müssen die Ergebnisse auch aufaddiert werden. Es handelt sich um eine Multiply-Accumulate Operation. Daher ist es notwendig im Design Speicherzellen vorzusehen, die schrittweise durch Addition vervollständigt werden, bis das Endergebnis vorliegt.

Der "Matrix-Kern" aus Speicherzellen für die Multiplikation ist in Abbildung 2.10 aus [11] dargestellt. Dieses Konstrukt wird auch Matrix Accumulator (MAC) genannt, da es Zwischenergebnisse aufsummiert, also kumuliert.

Wenn zwei Vektoren außen an den MAC angelegt werden, dann berechnet er ein Zwischenergebnis für das Outer Product aus den beiden Vektoren (siehe Formel (2.2)) und akkumuliert das Zwischenergebnis zum Rest auf (siehe Formel (2.3)). Wenn alle Vektoren angelegt worden sind, enthält der MAC somit das komplette Ergebnis der Multiplikation.

Die Kombination aus dem blockweisen Ansatz und dem Outer Product mittels MAC ermöglicht es ein Design zu erstellen, dass dem angestrebten RISC-V AME Ansatz entspricht.

Figure 1 4x4 Outer Product MAC Array with Local Accumulators

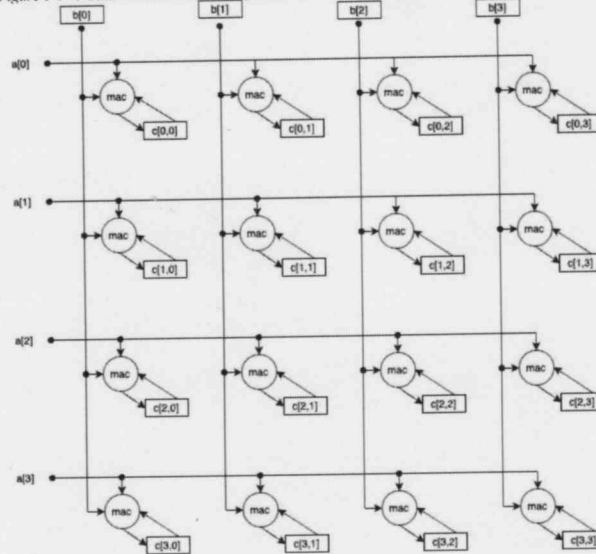


Abbildung 2.10: Matrix Accumulator (MAC)

2.9 Zusammenfassung

Dieses Kapitel über die theoretischen Grundlagen beginnt bei der mathematischen Definition von Matrizen und führt hin zu einer konkreten Umsetzung der Matrixmultiplikation in einem Computersystem. Die Gedankengänge aus diesem Kapitel stellen die Grundlagen für die Implementierung dar.

Da die streng mathematische Multiplikation auf einem Load-Store-Computersystem zu langsam ist, wurde die SIMD Verarbeitung in Kombination mit der Cache-Hierarchie besprochen. Auf einem RISC-V System gibt es bereits SIMD Instruktionen für die Berechnung von Vektoren in Form der Vektor-Extension. Es wurde beschrieben, wie das Hardware-Software Co-Design des Strip-Mining für die RVV-Extension aufgebaut ist.

Ausgehend von diesem Standpunkt sind die drei großen Strömungen IME, VME und AME der RISC-V Charter zur Handhabung der Matrixberechnung vorgestellt worden. Es wurde erläutert, welche Überlegungen zur Entscheidung zugunsten der AME für diese Bachelorarbeit zugrunde liegen.

Das Kapitel wurde mit der Beschreibung der blockweisen Berechnung der Matrixmultiplikation unter Verwendung des Outer-Produkt Ansatzes geschlossen. Damit ist das Strip-Mining auch für Matrizen möglich.

Gerüstet mit einem dokumentierten Zielbild kann als nächstes eine Implementierung methodisch geplant und dann durchgeführt werden.

keine
Leerzeilen
LD-
LV-
LD-

LV-

LT-1s

3 Durchführung und Methodik

Das Ziel ist beschrieben. Es geht um die Erweiterung eines RISC-V Kerns um eine hardwarebeschleunigte Matrixmultiplikation. Dabei soll die Arbeit möglichst nahe an einer der möglichen Erweiterungen liegen, die momentan tatsächlich von den RISC-V Gremien vorbereitet werden.

LV
LV-

Es wurde besprochen, mit welcher Technik die Matrixmultiplikation ausgeführt werden soll und welche theretischen Grundlagen hinter den Überlegungen stehen.

In diesem Kapitel wird beschrieben, welches Vorgehen gewählt wird um die Arbeit durchzuführen.

3.1 Schrittweises Vorgehen

Das Vorgehen besteht aus folgenden Schritten:

1. Analyse der theoretischen Grundlagen zur Matrixmultiplikation. Es soll eine Möglichkeit gefunden werden, die die Berechnung auf einem FPGA ermöglicht und mit VHDL beschrieben werden kann. Dieser Schritt erfolgte im Kapitel 2.
2. Analyse der NEORV32 CPU. Es sollen die Komponenten der CPU identifiziert werden mit dem Ziel, die Stellen zu finden, die geändert bzw. erweitert werden müssen. Dieser Schritt erfolgt in Kapitel 4.
3. Ein Compiler (selbst erstellt oder der GCC Compiler) wird verwendet um Testprogramme zu übersetzen.
4. Die Änderungen werden in den NEORV32 eingebracht.
5. Das geänderte Design wird simuliert bis die gewünschten Signale vorliegen.
6. Das geänderte Design wird synthetisiert und auf dem FPGA ausgeführt. Das Design wird angepasst, bis es auf dem FPGA funktioniert.
7. Die Testprogramme werden ausgeführt. Die Zyklenzahl wird gemessen und verglichen.

LI,

Das gewählte Vorgehen während der Änderung lediglich des Quellcode ist in Abbildung 3.1 dargestellt.

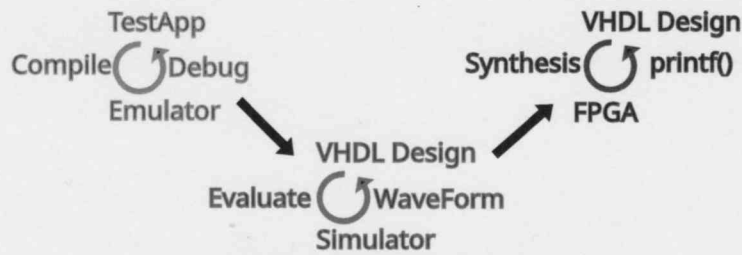


Abbildung 3.1: Entwicklungsschritte

Es gibt drei große Phasen, die in einem Wasserfallprozess streng linear abgearbeitet werden. Innerhalb jedes Schritts findet ein iteratives Vorgehen statt, dass durch den Kreis dargestellt werden soll.

Links des Kreises ist der Name des Vorgehens aufgeführt, das den Quellcode in den kompilierten Zustand überführt, also der Compiler, das Evaluieren oder Synthetisieren. Unterhalb des Kreises ist das Werkzeug aufgelistet, mit dem das Kompilat ausgeführt wird. Rechts des Kreises ist das Mittel aufgelistet, über das die Ausführung bewertet wird.

Der Quellcode wird bearbeitet und getestet, bis er das gewünschte Ergebnis erzielt. In diesem Fall wird in die nächste Phase übergegangen. Dies wird durch die Pfeile zwischen den Phasen angedeutet.

1. i
1. p
L 1.5
L 1.5
L 1.5

verändert

3.2 Der NEORV32 Core und dessen Einsatz

Der NEORV32 Chip ist ein VHDL- und auch ein Verilog-Design von Stephan Nolting und anderen Programmierern¹, welches als Open-Source Projekt unter der BSD-3 Lizenz veröffentlicht ist [14]. Es gibt zum einen sehr gute Dokumentation zu dem Projekt in Form von textueller Dokumentation, dem sogenannten Datenblatt [15]. Zum anderen ist auch der Quellcode selbst sehr gut dokumentiert und einheitlich formatiert und damit gut lesbar.

Am Anfang der Entwicklung wurden zunächst zwei Anwendungen erstellt, die auf dem NEORV32 nach der Erweiterung um eine Matrix-Engine laufen sollen. Eine der Anwendungen verwendet die Extension, die andere nicht um die Laufzeit einer Matrixmultiplikation mit und ohne Erweiterung messen zu können. Die Entwicklung der Anwendungen erfolgt mit Hilfe eines eigenen Compilers und auch des GCC Compilers. Die Ausführung findet auf einem eigens entwickelten Emulator statt. Dieser Schritt ist in Abbildung 3.1 auf der linken Seite dargestellt. Wenn beide Testprogramme im Emulator funktionieren, geht die Entwicklung in die nächste Phase, bei der das VHDL-Design angepasst wird.

L 2-
L 2-

Satzbau?
L 1.5

Zunächst ist zu sagen, dass man den VHDL- oder Verilog-Quellcode auf un-

¹<https://github.com/stnolting/neorv32/graphs/contributors>